

Aufgabe 1: AVL-Bäume**(10 Punkte)**

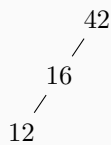
Fügen Sie die folgenden Zahlen nacheinander in einen *AVL-Baum* ein:

42 16 12 19 38 1

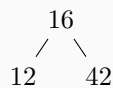
Zeichnen Sie den Baum vor und nach jeder durchgeführten Rotation. Geben Sie auch jeweils an, was für Rotationen Sie durchführen.

Lösung

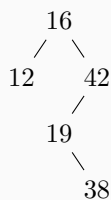
Einfügen von 42, 16, 12



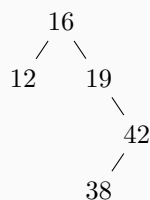
R-Rotation um 42



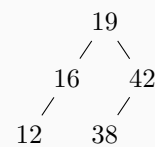
Einfügen von 19, 38



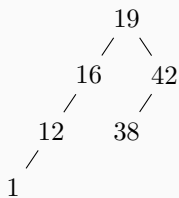
RL-Rotation um 16 (1)



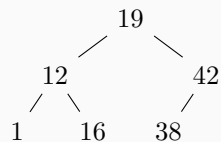
RL-Rotation um 16 (2)



Einfügen von 1



R-Rotation um 16



Aufgabe 2: Heaps**(10 Punkte)**

Fügen Sie die folgenden Zahlen nacheinander in einen *Min-Heap* ein:

45 18 19 7 13 2 22

Zeichnen Sie den Baum vor und nach jedem vollständigen Einfügen.

Lösung

Einfügen von 45

45

Einfügen von 18

18
/ 45

Einfügen von 19

18
/ 45 \ 19

Einfügen von 7

7
/ 18 \ 19
/ 45

Einfügen von 13

7
/ 13 \ 19
/ 45 \ 18

Einfügen von 2

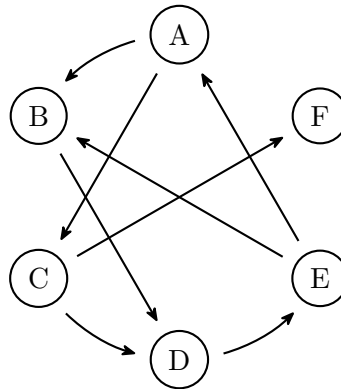
2
/ 13 \ 7
/ 45 \ 18 \ 19

Einfügen von 22

2
/ 13 \ 7
/ 45 \ 18 \ 19 \ 22

Aufgabe 3: Graphen**(10 Punkte)**

Betrachten Sie den folgenden gerichteten Graphen:



- Erstellen Sie die Adjazenzliste und die Adjazenzmatrix zu diesem Graphen.
- Welche der beiden Darstellungen ist für diesen Graphen effizienter? Begründen Sie Ihre Antwort.

Lösung**Adjazenzliste:**

- A: B, C
- B: D
- C: D, E
- D: E
- E: A, B
- F: -

Adjazenzmatrix:

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	0	0	0	1	0	0
C	0	0	0	1	0	1
D	0	0	0	0	1	0
E	1	1	0	0	0	0
F	0	0	0	0	0	0

Effizienz: Die Adjazenzliste ist für diesen Graphen effizienter, da sie nur die vorhandenen Kanten speichert. In diesem Fall gibt es 8 Kanten, während die Adjazenzmatrix 36 Einträge (6x6) hat, von denen nur 8 den Wert 1 haben.

Generell bietet die Adjazenzmatrix eine konstante Zeitkomplexität für die Überprüfung, ob eine Kante existiert, während die Adjazenzliste im schlimmsten Fall linear zur Anzahl der Kanten sein kann. Bei diesem eher dünn besetzten Graphen müssen dafür aber pro Knoten sehr viel mehr Einträge in der Matrix gespeichert werden, was neben dem Speicherbedarf auch bspw. das Iterieren über alle Nachbarn eines Knotens ineffizient macht.

Aufgabe 4: Datenstrukturen**(10 Punkte)**

Erläutern Sie die Idee und Funktionsweise des folgenden Datentyps. Diskutieren Sie kurz die Vor- und Nachteile gegenüber ähnlichen Datentypen aus der Vorlesung.

```
1  type FooType struct {
2      values    []int
3      children  []*FooType
4  }
5
6  func (f *FooType) Add(value int) {
7      if len(f.values) < 2 {
8          f.values = append(f.values, value)
9          return
10     }
11     if value < f.values[0] {
12         f.AddToChild(0, value)
13         return
14     }
15     if value < f.values[1] {
16         f.AddToChild(1, value)
17         return
18     }
19     f.AddToChild(2, value)
20 }
21
22 func (f *FooType) AddToChild(i, value int) {
23     for len(f.children) <= i {
24         f.children = append(f.children, &FooType{})
25     }
26     f.children[i].Add(value)
27 }
```

Lösung

Der Datentyp `Footype` ist ein *ternärer Suchbaum*. In jedem Knoten des Baums werden bis zu zwei Werte gespeichert und es gibt bis zu drei Kindknoten. Die Werte im linken Kindknoten sind kleiner als der erste Wert im Elternknoten, die Werte im mittleren Kindknoten sind größer als der erste Wert im Elternknoten und kleiner als der zweite Wert im Elternknoten, und die Werte im rechten Kindknoten sind größer als der zweite Wert im Elternknoten.

Die Funktion `Add` fügt einen Wert in den Baum ein. Dazu benutzt sie die Funktion `AddToChild`, die einen Wert in einen Kindknoten einfügt und diesen ggf. vorher erzeugt. Die Funktion `AddToChild` ruft dann wieder `Add` auf, um den Wert in den Kindknoten einzufügen.

Die Datenstruktur ist sehr ähnlich zu einem binären Suchbaum, allerdings wird der Baum breiter und flacher, da jeder Knoten bis zu zwei Werte speichert. Dadurch ergibt sich ein minimal besseres Laufzeitverhalten.

Die Komplexitätsklasse der Operationen ist die gleiche wie bei einem binären Suchbaum. Der Nachteil ist, dass die Datenstruktur wesentlich unübersichtlicher ist. Sobald bspw. eine Funktion zum Löschen hinzu kommt, kann es passieren, dass ein Knoten nur noch einen Wert enthält, dieser aber nicht der linke der beiden Werte ist. Dann muss der Baum umorganisiert werden oder beim Einfügen muss darauf geachtet werden, dass auch Werte ungültig sein können.

Aufgabe 5: Komplexität**(10 Punkte)**

Betrachten Sie die folgende Funktion `SameElements()`. Die Funktion erwartet zwei `int`-Listen und prüft, ob diese beiden Listen die gleiche Menge an Elementen enthalten. D.h. ob jedes Element aus der einen Liste auch in der anderen enthalten ist. Dabei spielt es keine Rolle, ob die Länge der Listen gleich ist bzw. ob die Elemente in der gleichen Anzahl vorkommen.

```
1 func SameElements(list1, list2 []int) bool {
2     for _, v1 := range list1 {
3         contained := false
4         for _, v2 := range list2 {
5             if v1 == v2 {
6                 contained = true
7             }
8         }
9         if !contained {
10            return false
11        }
12    }
13    for _, v2 := range list2 {
14        contained := false
15        for _, v1 := range list1 {
16            if v1 == v2 {
17                contained = true
18            }
19        }
20        if !contained {
21            return false
22        }
23    }
24    return true
25 }
```

- Bestimmen Sie die Komplexität dieser Funktion. Geben Sie in O-Notation an, wie oft die Vergleiche `if v1 == v2` durchgeführt werden. Begründen Sie Ihr Ergebnis.
- Machen Sie einen Vorschlag, wie diese Funktion optimiert werden kann. Erläutern Sie, inwiefern dieser eine bessere Komplexität hat.

Hinweis: Sie müssen keinen konkreten, vollständigen Algorithmus angeben.

Lösung

- Die Funktion liegt in $O(n \cdot m)$, wobei n und m die Längen der beiden Listen sind. Es gibt zwei geschachtelte Schleifen, wo jeweils innerhalb eines Durchlaufs durch eine Liste für jedes Element ein Durchlauf durch die andere Liste passiert. In jeder dieser Schleifen werden daher $n \cdot m$ Vergleiche durchgeführt.
- Eine Optimierungsmöglichkeit wäre, die beiden Listen zu sortieren, Duplikate zu entfernen und die Listen dann auf Gleichheit zu testen. Das Sortieren liegt in $O(n \cdot \log n + m \cdot \log m)$, Duplikate entfernen und der exakte Vergleich von Listen haben jeweils lineare Laufzeit. Da diese Aktionen hintereinander geschehen und nicht geschachtelt werden, bleibt es bei der Komplexität des Sortierens, also insgesamt bei $O(n \cdot \log n)$, wobei n die Länge der größeren Liste ist.